CHECK_TIME_DISTANCE

Overview

check_time_distance verifies that start_event is followed by test_event, and the time distance between start event and test_event falls within predefined time distance range. Not before a minimum of min_time and a maximum of max_time timing period from the start event it's test_event must be true. Note, that time distance parameters are expressed in the simulation time units. Also note that this checking component does not requires to sample the system clock.

Check_time_distance generates an error messages as well as asserts on of the two access events (err an ok) to communicate with the upper-level components.

Syntax

```
check_time_distance #(min_time,max_time) InstName (start_event,
test_event);
```

Parameters	
min_time	Minimum distance between starts and test events (in simulation time units)
max_time	Maximum distance between starts and test events (in simulation time units)
Interface	
start_event	1-bit signal defining start event
test_event	1-bit signal defining test event
Access Events	
err, ok	Signal about failure or success. Can be monitored externally for test control purposes.
Defines	

Usage

check_time_distance complements clock-dependent timing checks with the time-dependent checks. In many design specifications, design requirements are expressed with the actual time units, and not with the system clock cycles. Check_timr_distance verification component allows to directly formalize such requirements for the specification purposes.

Verilog Example

```
module test;

reg clk;
reg a, b;

//--------------------------------------------------
function integer rn;
input d1, d2;
integer d1, d2;

begin
  rn = d1 + {$random} % (d2-d1+1);
end

endfunction

//--------------------------------------------------
check_time_distance #(80,140) distance (a,b);
//--------------------------------------------------

initial begin
  clk = 0;
  a   = 0;
  b   = 0;
end

always #5 clk <= ~clk;

//--------------------------------------------------
always @(posedge clk) begin
  if (rn(0,100) < 4) a <= 1;
  else               a <= 0;
  if (rn(0,100) < 4) b <= 1;
  else               b <= 0;
end


always @(distance.err) $display ("Test: wrong distance");
always @(distance.ok)  $display ("Test: right distance");

initial begin
  $monitor ("a = %b, b = %b, time = %0t", a, b, $time);
  #3000 $finish;
end

endmodule
```